

截图 1：SGLang OpenAI 兼容服务 — 访问 /v1/models 并完成一次推理请求

```
HW1 截图 1 — SGLang OpenAI 兼容服务 /v1/models 与一次推理请求

$ curl -s http://127.0.0.1:30000/v1/models | python3 -m json.tool
{
  "object": "list",
  "data": [
    {
      "id": "/workspace/models/Qwen3-0.6B",
      "object": "model",
      "created": 1790965009,
      "owned_by": "sglang",
      "root": "/workspace/models/Qwen3-0.6B",
      "parent": null,
      "max_model_len": 40960
    }
  ]
}

$ curl -s http://127.0.0.1:30000/v1/chat/completions -H "Content-Type: application/json" \
-d '{"model": "/workspace/models/Qwen3-0.6B", "messages": [{"role": "user", "content": "用一句话解释什么是 KV Cache"}], "max_tokens": 96, "temperature": 0}' | python3 -m json.tool
{
  "id": "50776e6ec699403ca7bb74f1a3620866",
  "object": "chat.completion",
  "created": 1790965016,
  "model": "/workspace/models/Qwen3-0.6B",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "<think>\n嗯，用户问的是“用一句话解释什么是 KV Cache”。首先，我需要确定用户的需求是什么。可能他们是在学习计算机科学，或者在使用某个系统时遇到了相关概念，想要一个简洁的解释。接下来，我得回忆一下KV Cache的基本定义。\n\nKV Cache通常指的是Key-Value Cache，也就是键值缓存。它的作用是存储频繁访问的键值对，以加快后续访问的速度。不过，我需要确认这个"
      },
      "logprobs": null,
      "finish_reason": "length",
      "matched_stop": null
    }
  ],
  "usage": {
    "prompt_tokens": 14,
    "total_tokens": 110,
    "completion_tokens": 96,
    "prompt_tokens_details": null,
    "reasoning_tokens": 0
  },
  "metadata": {
    "weight_version": "default"
  }
}
```

截图 2：Mooncake trace 采样 workload — 24 条请求、Poisson 到达、逐请求 TTFT/latency 记录

```
HW1 截图 2 — Mooncake trace 采样 workload 回放 (24 条请求, Poisson 到达)

$ python3 mooncake_workload.py --num-requests 24 --mean-interval 18.0
# 从 Mooncake FAST'25 arxiv-trace (23,608 条) 中采样 24 条真实记录
# (input_length<=2048, output_length<=128 窗口), 按 input_length 构造 synthetic prompt,
# Poisson 到达 (指数间隔, 均值 18s), 发送到 SGLang OpenAI 兼容服务并逐请求记录指标
[trace] 候选记录 569 条, 采样 24 条 (input<=2048, output<=128)
[00] send= 7.79s status=200 in=1009 out=39 ttft=2.154 latency=7.075
[01] send= 36.24s status=200 in=1443 out=38 ttft=4.171 latency=14.041
[02] send= 58.02s status=200 in=1454 out=116 ttft=1.677 latency=33.065
[04] send= 59.77s status=200 in=894 out=8 ttft=2.059 latency=3.597
[03] send= 62.24s status=200 in=1340 out=100 ttft=1.331 latency=26.11
[05] send= 89.86s status=200 in=1295 out=16 ttft=3.064 latency=5.28
[06] send= 99.95s status=200 in=994 out=20 ttft=2.132 latency=4.557
[08] send= 117.12s status=200 in=903 out=32 ttft=3.361 latency=8.971
[07] send= 131.95s status=200 in=898 out=89 ttft=3.521 latency=24.138
[10] send= 136.87s status=200 in=1193 out=35 ttft=2.598 latency=10.14
[09] send= 137.63s status=200 in=895 out=77 ttft=1.919 latency=19.283
[11] send= 184.29s status=200 in=1524 out=96 ttft=3.535 latency=23.072
[12] send= 193.60s status=200 in=908 out=102 ttft=2.205 latency=18.2
[14] send= 223.41s status=200 in=897 out=22 ttft=2.15 latency=12.423
[15] send= 224.49s status=200 in=1926 out=18 ttft=4.797 latency=10.006
[13] send= 227.11s status=200 in=957 out=110 ttft=1.988 latency=26.365
[16] send= 257.09s status=200 in=911 out=52 ttft=1.899 latency=8.305
[17] send= 284.99s status=200 in=1445 out=14 ttft=3.221 latency=5.326
[18] send= 298.61s status=200 in=918 out=23 ttft=1.882 latency=10.491
[20] send= 305.93s status=200 in=1312 out=1 ttft=3.075 latency=3.075
[19] send= 313.33s status=200 in=1622 out=91 ttft=3.954 latency=23.407
[21] send= 325.55s status=200 in=1062 out=61 ttft=2.19 latency=10.36
[22] send= 362.04s status=200 in=973 out=115 ttft=2.047 latency=20.224
[23] send= 363.03s status=200 in=1058 out=13 ttft=2.85 latency=4.843
===== WORKLOAD SUMMARY =====
{
  "num_requests": 24,
  "num_success": 24,
  "success_rate": 1.0,
  "wall_clock_s": 363.029,
  "mean_interval_s": 18.0,
  "total_input_tokens": 27831,
  "total_output_tokens": 1288,
  "output_token_throughput_tps": 3.548,
  "ttft_mean_s": 2.658,
  "ttft_p50_s": 2.197,
  "ttft_p95_s": 4.138,
  "latency_mean_s": 13.848,
  "latency_p50_s": 10.425,
  "latency_p95_s": 26.327
}
CSV: /workspace/HW1/results/workload_results.csv
```